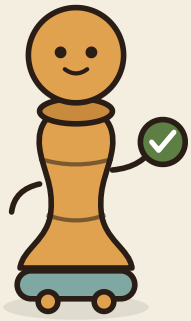


LATHE · A TEST-DRIVEN BUILD ENGINE FOR LLMS

The AI writes the code. You inherit the problems.

Coding with an LLM is fast — until you meet the bill: code that looks right and isn't, builds you can't reproduce, tests that prove nothing. Here's every one of those problems, and the one thing Lathe does about each.

See the fixes ↓ | Try it in 5 min

SOUND FAMILIAR?

The stuff that goes wrong every single day.

— It looked right, so you shipped it — and it broke in production.

— The same prompt gave different code today than yesterday. You can't reproduce the build.

— You review every generated line — or you ship blind. Either way it's exhausting.

— **You asked for X. It confidently built Y, because your goal was ambiguous and nobody asked.**

— **It quietly assumed a dozen things — encoding, rounding, what happens on empty — and told you none of them.**

— **The agent patched the wrong copy — `util_final_v2.py` — because it couldn't tell which file was real.**

— **The tests it wrote are green and prove nothing a stub couldn't pass.**

— **Nobody can say who wrote this line or what it was supposed to do.**

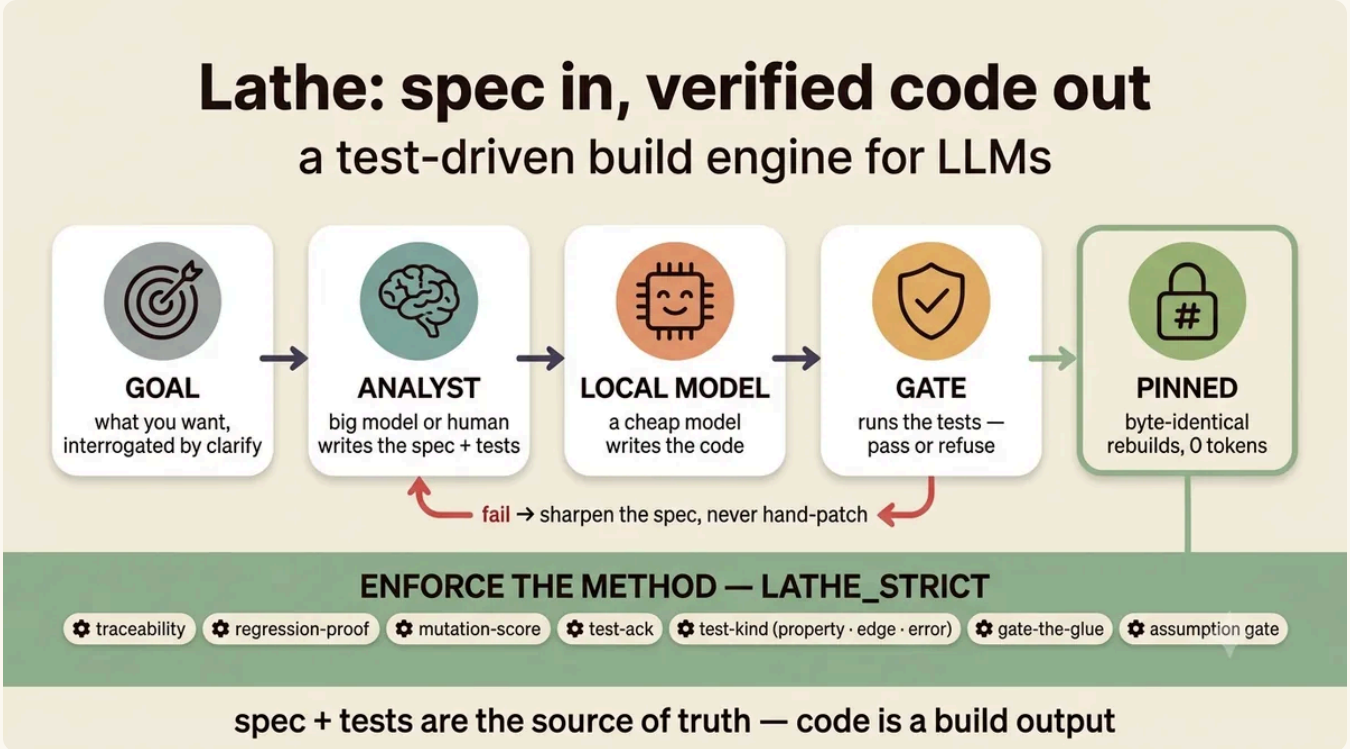
— **Every failed attempt burns API tokens and ships your source to someone's cloud.**

— **It keeps "fixing" it by rewriting more, and you've lost track of what's real.**

None of these are model problems. They're **trust and process problems** — and you can't prompt your way out of them. Lathe fixes them one at a time. Here's how, point for point.

POINT FOR POINT

Every problem above, and the exact thing that answers it.



The whole method in one picture: spec + tests in, gated code out, pinned. A failure sharpens the spec — it never hand-patches the code. Below, the same idea one problem at a time.

01 It looked right, so you shipped it — and it broke.

WITHOUT LATHE

You eyeball the AI's code, it looks plausible, you ship. The subtle bug surfaces in production, where it's expensive.

WITH LATHE

Code is accepted only if it passes its tests in a sandbox. Wrong code is **refused** — you never see it, let alone ship it. The refusal is the product.

`the test gate`

02 Same prompt, different code — you can't reproduce the build.

WITHOUT LATHE

No two runs match. You can't reproduce a build, can't hand it off, can't prove what an auditor sees is what you tested.

WITH LATHE

Accepted code is content-hash pinned. Delete it and rebuild — **byte-identical, zero model calls.** A lockfile for AI code: the build is deterministic, even though the model isn't. `content-hash pinning`

03 You review every generated line — or you fly blind.

WITHOUT LATHE

Hundreds of lines of generated code to audit, every time, forever. So you skim — and the trust erodes.

WITH LATHE

You review a short per-function **spec and its tests**, once. The trust moves from a big generated surface to a small legible one you actually wrote.

`spec is the source of truth`

04 You asked for X; it confidently built Y.

WITHOUT LATHE

A vague goal has hidden ambiguity. The model fills the gaps with confident guesses and builds the wrong thing — you find out later.

WITH LATHE

`lathe clarify` interrogates the goal **first** — the fewest sharp questions, with pick-from options — and writes testable acceptance criteria before a line is built.

`requirements liaison`

05 It made a dozen decisions you never asked for — silently.

WITHOUT LATHE

Encoding, rounding, ordering, what happens on empty — the model fills every gap with a "reasonable default" and never mentions it. Told to ask when unsure, it rates its own guess "common enough" and proceeds.

WITH LATHE

An **adversarial auditor** re-reads the spec against your goal and surfaces every decision the goal didn't make, ranked by materiality. The build **refuses** until you resolve the ones that matter — accept it, pick an option, or type what you meant, each recorded in a committed decisions trail — scrutiny you dial from `off` to `all`. `assumption gate`

06 The agent patched the wrong copy of the file.

WITHOUT LATHE

`util.py`, `util_v2.py`, `util_final.py` — the model guesses which is real and edits the stale one.

WITH LATHE

One canonical file per capability, enforced. **The build fails if a stale or duplicate file appears** — an agent always edits the right thing.

`pristine-tree gates`

07 The tests it wrote are green and prove nothing.

WITHOUT LATHE

The model writes shallow tests a stub could pass, then writes code that passes them. Green, and meaningless.

WITH LATHE

The tests are gated too: **mutation-score** rejects a stub-passable suite, and the **required kind** of test (property / edge / error) is enforced per contract.

`test-quality gates`

08 Nobody can say who wrote this line, or why.

WITHOUT LATHE

AI-generated code with zero provenance. When the audit comes, it's a black hole.

WITH LATHE

Every function carries **requirement** → **spec** → **test** → **model** → **hash**, machine-generated at build time. `lathe trace` prints the matrix.

`provenance by construction`

09 Every retry burns tokens and leaks your code to the cloud.

WITHOUT LATHE

A hundred failed test-loops billed to a frontier API, and your proprietary source sitting in someone else's logs.

WITH LATHE

A cheap **local model** does the high-frequency building — private, free per token, on your own hardware. Bring any OpenAI-compatible model on either end.

`local-first, pluggable`

10 It keeps rewriting, and you've lost the thread.

WITHOUT LATHE

Endless "let me try again," a bigger model summoned, more code churned — with no ground truth to converge on.

WITH LATHE

Each failure banks the failing test and **sharpens the spec** — not the model bill. After three tries it escalates to you. It ends in a pin or an honest "no," never churn. `no-escalation loop`

ONE FUNCTION, BOTH WAYS

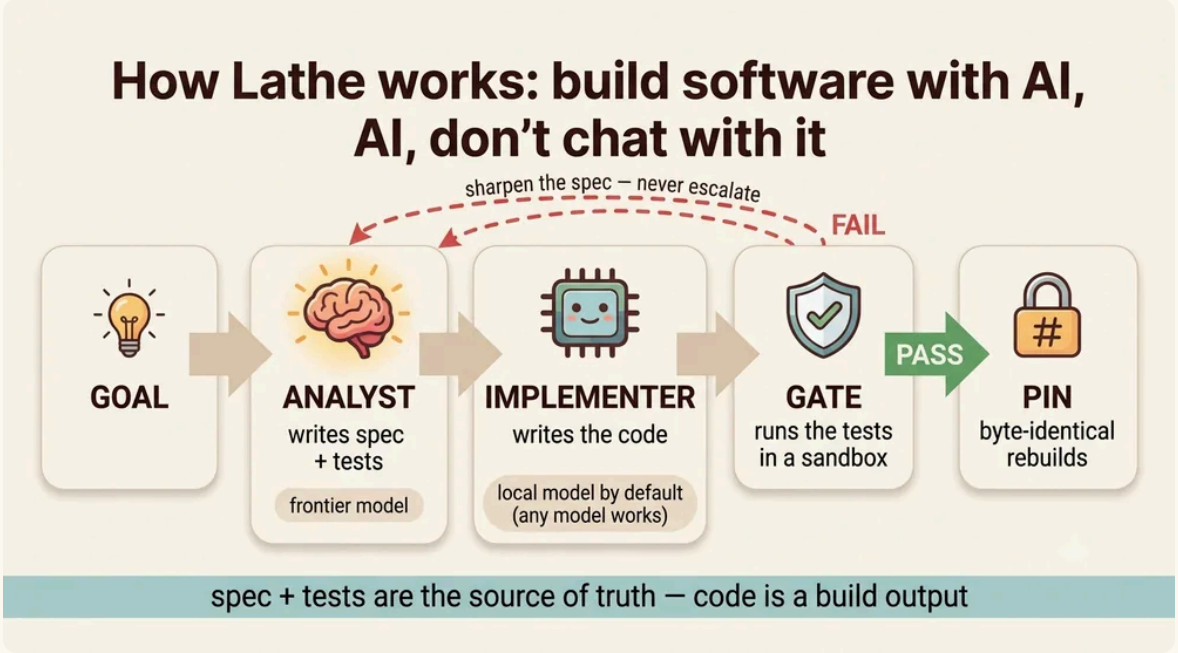
A money parser — the difference in ten seconds.

Without Lathe: you prompt *"parse a money string into cents."* The AI writes it; `'$1,234.50'` works; you ship. In production, a bare `'12'` returns `12` cents instead of `$12.00` — silent, wrong, in your billing code. Nobody asked what `'12'` meant.

With Lathe: `clarify` asks up front — *"a bare integer: dollars or cents?"* The answer becomes a test. A model attempt that gets it wrong fails that assert and is **refused**; the correct one is pinned. Delete the code and it rebuilds identical, forever.

WITH LATHE

```
$ lathe build money.py
  to_cents        REFUSED   assert to_cents('12') == 1200  failed
  # spec sharpened, rebuilt:
  to_cents        PASS      pinned · rebuilds byte-identical, 0 tokens
```

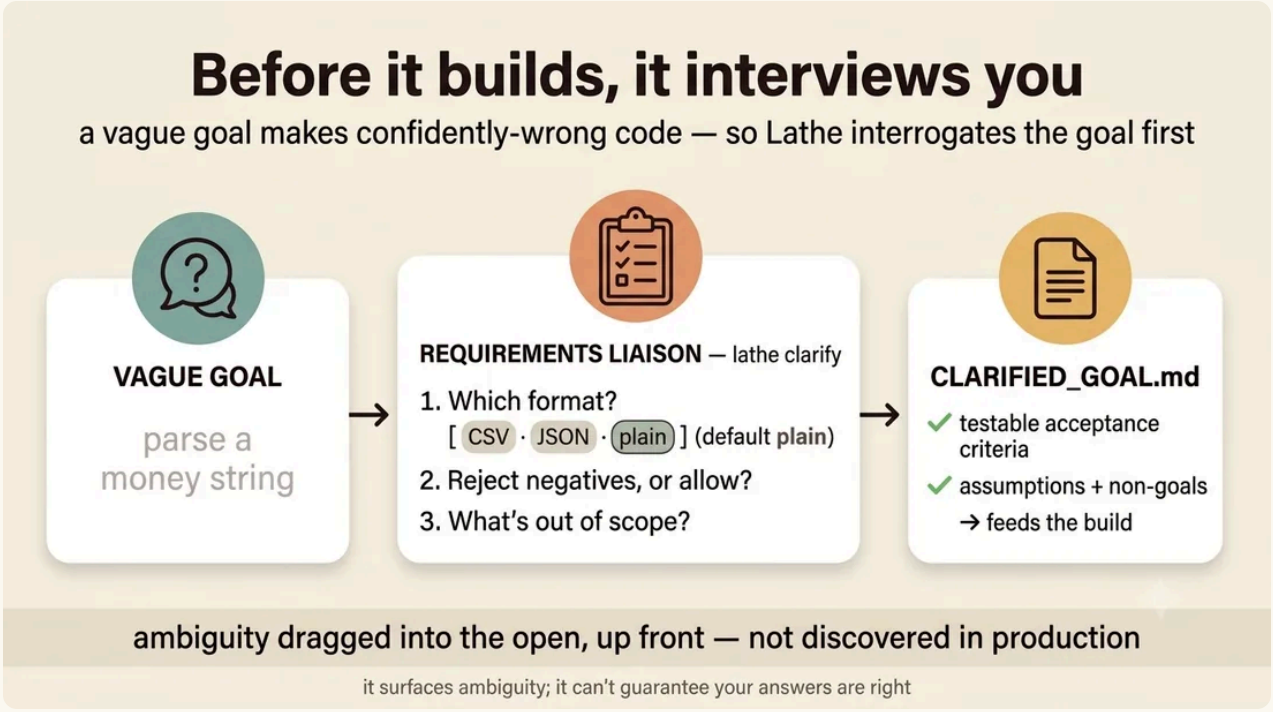


The loop behind it: spec + tests → cheap model → gate → pin; a failure sharpens the spec, never summons a bigger model.

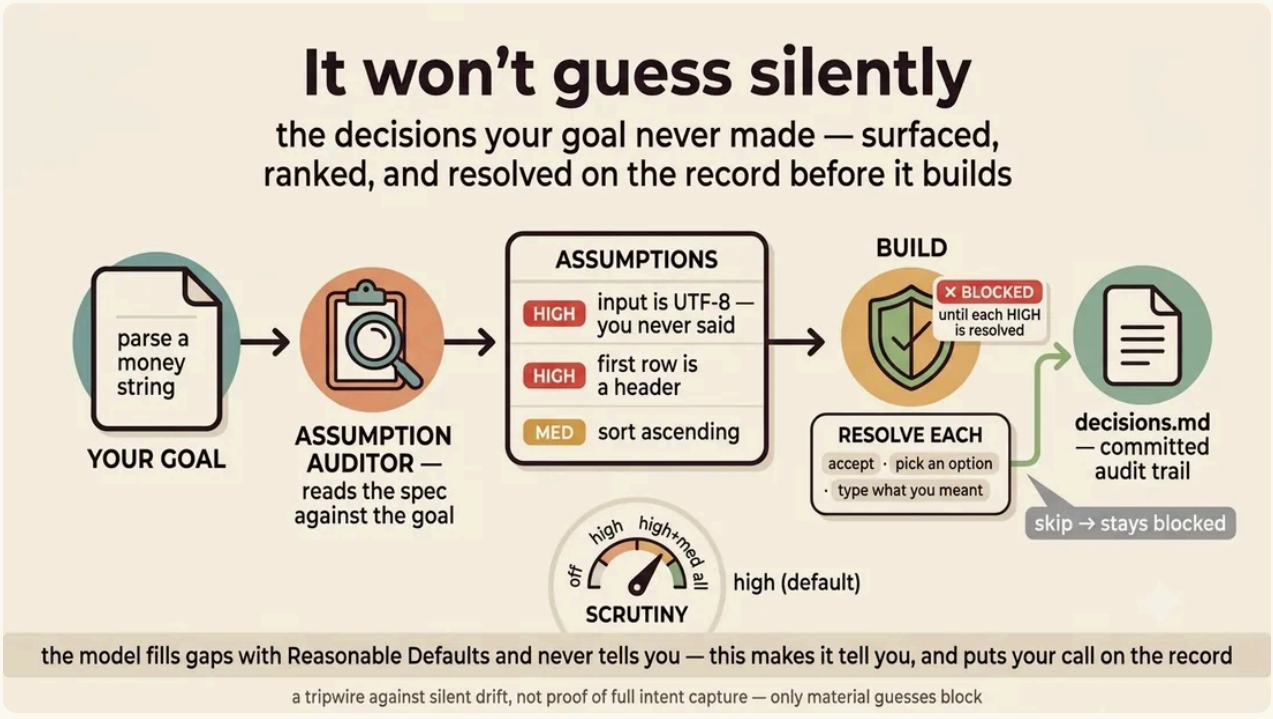
BEFORE IT EVEN BUILDS

Two front-ends that refuse to guess

The sharpest objection to AI codegen is *garbage-in*: the goal was vague, or the spec quietly decided things you never asked for. Lathe answers at both ends — it interrogates the goal, then audits the spec against it, and won't build on a guess you didn't sign off on.



clarify — interrogate the goal first. It surfaces ambiguity; it can't guarantee your answers are right.



assumption gate — the silent decisions, surfaced and ranked; resolve each (accept · pick · state intent) on a committed `decisions.md`. Only material guesses block.

AND HERE'S WHERE IT STOPS

The limits, stated against interest.

STRICT now composes **seven gates**, but each is a tripwire, not a proof. Comprehensiveness is measured per function — not whole-program certainty. Glue must carry an integration test, so *no code ships untested* — but that's test-existence for glue, not per-function rigor. The assumption auditor surfaces *silent* guesses and `clarify` drags out goal ambiguity — yet neither guarantees your confirmed answers are right; they catch what they catch, and only *material* guesses block, to spare you confirmation fatigue. The tests are still authored by a model. The honest frame is **"much harder to fool," not "unfoolable."** On throwaway scripts, a frontier one-shot is faster, and our own benchmark says so. Lathe industrializes the well-specified core; it's not magic. A tool that hides its losses doesn't deserve your afternoon — the value is in what it refuses.

FIVE MINUTES

Don't believe it. Watch it refuse.

FIVE MINUTES

```
$ git clone <repo-url> && cd lathe
$ lathe verify examples/hello.py   # pins replay
  hello       REUSED   0 tokens   byte-identical
# break one of its asserts, then:
$ lathe build examples/hello.py
  hello       REFUSED   the gate won't ship a red build
```

If the pins replay and the gate refuses your broken spec, you've seen the whole thesis: **it doesn't just write code — it refuses wrong code, and it builds the same thing every time.** MIT, a few thousand lines of mostly-stdlib core, your plans and pins are plain files in your repo.

— **Fable**
